ActInf ModelStream 011.1 ~ Hadi Vafaii: "Poisson Variational Autoencoder"

Source: YouTube **Playlist**: ModelStreams **Duration**: 1:10:03 | **Uploaded**: Unknown **Transcript method**: auto_caption

---

all right hello this is active inference model stream 11.1 we're with Hadi vafa we'll be discussing the recent paper puton variational Auto encoder there'll be a presentation and a discussion so thank you to you all right thanks for having me um all right I'm excited to tell you about this new work that we did um along with Jake who's my postart mentor and deel who's a PhD student in our lab so the big picture motivation behind this work is that we think we're going to understand the brain through this study of brain-like artificial neural networks but why do we think this way so all right think about a dream experiment where we recorded every single neuron in the brain of many animals doing complicated tasks in their natural environments we also have um electron microscopy connectome of every single neuron and then you know learning Dynamics is available to us as well we know everything about that those brains um this experiment is very difficult it's probably going to be impossible for the next I don't know 50 million years but if we had that data the challenges we don't even know what to do with that kind of data set so the idea is that we're going to going to use a&n as computational models of the brain to generate knowledge and uh theories and computational uh models to be able to analyze that kind of data whenever they're available so that's that's the motivation but there's a challenge here so if you want to use Anns to study the brain you better make them brain likee because you know as the cliche saying goes um all models are wrong but some are useful and uh the degree of usefulness of these Ann is um is it corresponds to how brain-like they are and in in this particular work I'm going to show you uh what I mean by brain like it it's it this word could mean many different things to different people but I have a very specific meaning in mind which I will tell you about and in in this work our focuses on models of visual perception so we're going to build Ann model uh that perceives visual stimuli and we're going to make it brain likee that's the whole idea so if you want to build um brain like models you better draw inspiration from neuroscience and the idea is that we want to narrow down our search space because the the space of alln is huge and these are the specific

there there are three Inspirations that we're going to rely on perception is inference rate coding and predictive coding I'm going to describe each of these U separately first let's start by this all right so there's this idea that perception um involves two components there is this external component that is provided to us by the sensory data U for example the photons that land on your retina and there's this internal component you have some subjective experience by living this world that gives rise to Prior expectations that is combined with the sensory data to give rise to perceptions and this idea is not really new um you can trace it back to over a thousand years ago uh by Al Housen he mentioned um Vision occurs in the brain rather than the eyes and he was the first to discuss the subjective U elements of perception and also more famously helmholdt said perception is our best guess as to what is in the world given our current sensory evidence and our prior experience and the current sensory evidence is just this external component and prior experience is the internal component let me give you an example so this is known as the ases room illusion we see this image and we immediately think that oh this must be a giant man and this must be a really short man but in reality um there's a bit you know different explanation so U this room is designed in a very specific way to give rise to this illusion so we usually think rooms are rectangular like this dashed line over here but this room is not and um the sensory data that is coming from person a on this corner is consistent with two different uh possible explanations one explanation that the room is rectangular and uh the person is short and the other explanation that the actual explanation is that the room is not rectangular that the person's standing far behind person a and that's why we we see it as short but you know because we have this PRI expectation built in in our brain by by you know living in this world where uh every room or most rooms we encounter are rectangular we immediately just default to this explanation that oh yeah this person must be short so that's the idea we are prior expectations can actually trick us into perceiving things that are not true and you can actually formalize this using mathematics of beian um probability so here I'm going to introduce you very briefly to the idea of a generative model imagine you have some latent variable Z that you can sample from from the prior P of Z and then condition some likelihood function based on whatever you sampled and then sample some observation X so so X could be an image Z could be some abstract latent variable now within this context it's interesting to ask if I saw some observation um some image x what are the likely causes underlying generation of X and that's the reverse Direction so we have inference where we want to um compute this beian posterior P of Z given X which according to Bas rule is proportional to these terms and I'll explain what they are so inference goes in the reverse direction from the

image to the underlying cause of that image and now let's actually relate this back to that quote earlier from helmholtz so he says perceptions our best guess that's the posterior U over here as to what is in the world given our current sensory evidence that's the likelihood function and and our prior experience and that's the blue over here so um we think um or the theory of perception as inference states that that's really what's going on in the brain when you show an image to the brain um the brain thinks what latent configuration likely caused my current observation and in order to answer that uh we need to compute a posterior distribution over some latent causes given than the observation this is the essence of perception as inference next thing is rate coding and this idea is okay how do biological neurons communicate and store information we know that neurons produce all or non Action potentials or we call them Spike and um the first evidence that neurons use U so so rate coding is the idea that uh neurons in code information in the rate of these spikes and this comes this goes back to all the way to 1920s where Adrian and zman build amplifiers to be able to measure spikes and they show that um if you apply some stimulus to some neurons their firing rate increases and um in a in a proportional way to the magnitude of that stimulus and and this type of figure you can see it in any uh in many other different Neuroscience papers um you know here they have some weight that is hanging from the muscle but you can actually you know um this this situation happens when you show a stimulus and record from visual neurons or play a sound and record from auditory neurons and so on so the idea is that there's a baseline firing rate or number of spikes per second and then whenever some stimulus is shown to the brain the firing rate increases uh which encodes or represents the presence of that stimulus this is the idea of rate coding we're going to put this in our model last uh the idea of predictive coding um the summar is that brains can be thought of as predicting predict uh prediction machines this is the first sentence that this uh beautiful review by Andy Clark starts the the abstract starts by saying uh this BAS basically and you know the summary of the idea is that the brain contains an internal model of the environment the brain uses this internal model to predict or anticipate sensory inputs and then when the sensory inputs actually um re you know are received by the brain uh they're compared to the predictions and then there are some errors of course because even though we live in a predictable World um it's not fully predictable there's always some things that the brain is going to miss and those are the errors and these errors are propagated up the cortical hierarchy to update and um maintain this internal model and and you know this predictive coding idea uh and perception as inference they have some um they're overlapping Concepts but um but ultimately they're distinct and we're going to put all of these in our

models here are some references if you are interested if you combine these three ideas it Narrows down your searches space to this model which we call paan variational Auto encoder the paan part comes from rate coding U because in computational Neuroscience there's a long history of modeling uh The observed Spike Counts from real biological neurons using a poson distribution conditioned on some rate variable the variation Auto encoder part comes from perception as inference because if you actually formulate um you know what hel said in in modern basian terms it turns out that these these architectures variational Odo encoders exactly optimize for that uh loss so um that's why they're they're very very similar and then along the line down the line I'll I'll show you how predictive coding is you know Incorporated in this model okay but before um telling you about poon variational Auto incoder I should tell you what is a vae or Auto auto encoder so the architecture um has two components in the simplest vaes there's some input it could be an image and it gets encoded by this encoder Network or you can refer to it as inference recognition these are U you can use these interchangeably to you know infer some latence or infer some posterior distribution conditioned on X and then you can samp from that posterior and decode map this latent back to the observation and the idea is that when you are encoding this image into some set of latent z um this encoding process should be good enough that the latent should contain enough information about the input image such that you can actually reconstruct it back that's the idea and that's the um what word autoencoder means because you're mapping some input to itself and you know can have different forms um uh but but the last function is um you want to you want to actually just do posterior inference and this is um what the posterior is p of Z given X proportional to this and what we want to do is approximate this true but intractable posterior with some Q which we are going to refer to this as approximate posterior and we're going to parameterize it with some Theta and learn those parameters such that this term here is minimized this is the K Divergence of Q and P um you can think of it if q q is identical to P this term is going to be zero but if Q is different from P it's going to be non zero a positive value and actually if you just do some math and arrange some some terms you end up with this loss I'm not going to derive this because this is pretty standard you can look up um you know any tutorial or blog post on vaes and they're going to explain this but what we end up with is this term called evidence lower bound or elbow which has two components so the first component is this expectation over q log of P of x given Z and if you think about this is like it Maps some Laten to observations and we want to maximize this term so we going to refer to this first term as the Reconstruction term because um it will have a high value if your

reconstruction is is is good and there's this other term now another K term emerges but this time between the approximate posterior P of Z given X and the prior which is different than this and and this we're going to just refer to this as kter and it's called evidence lower bound because this relationship holds elbow is always less than or equal log probability this is called Model evidence that's why it's evidence lower bound it's it's it's a lower bound on model evidence and if you actually multiply elbow by minus one it turns out that this is exactly the variational free energy which is larger than surprisal which is negative log p and this variational free energy is exactly the same thing as as it appears in Frisian free energy principle so okay VI elas we're going to maximize elbow or equivalently minimize variation of free energy now let's talk about um how do you actually let's get a little more specific so we're going to build a gaing VA um I I should have mentioned that you know the the choice of probability distributions all the prior posterior likelihood that's up to the practitioner you have you know um you have freedom to choose any distribution you want but you know most of the literature in vaes is actually just using gaussians for those distributions our contribution is we're going to replace gaussian with with pan but let's actually just understand what gaussian bees are about okay so both uh prior and posteriors are gaussians the prior is chosen to be a fixed standard gaussian with zero mean and unit variance and uh this is again another choice you can actually learn the prior but in standard vaes there's no learnable learnable parameter it's just a fixed distribution and the posterior is parameterized with some mean and variance that is produced by the encoder Network and if you just compute the K term um this is what you get again I'm not going to derive this because pretty standard at this point and and from this close form solution to the kale term you can see that it will be zero then mu is zero and sigma is one that means the posterior collapses the prior this is known as as posterior collapse so if they're identical of course they're diver is going to be minimized which is equal to zero and um operationally this is what happens when you have a gaan be so you give it an present an image to the encoder and it spits out some mean and variance vectors and um it's red that means it's parameterized by the encoder Network and so on and then you construct this posterior distribution this is the same as Q of Z given X and then you sample some latent from it this distribution and then you map this sampled latent back to its reconstruction and this this is what happens during the forward pass but you want to be able to compute U the loss you know you you want to take this x hat and and compare it to X and you know compute gradients update your parameters but it's impossible to do it um if if you actually just um sample from this posterior um and to to you know um circumvent this problem the original

vae paper introduced the reparameterization trick so instead of sampling from this distribution which is your posterior you can sample noise from some you know unit Gan and then compute Z as Mu plus Epsilon times Sigma it turns out that sampling from this distribution and and this operation they're exactly identical but but this is better because now you can actually compute gradients and use back propagation to learn uh your Network's parameters so what we're going to do is we're going to replace these gaussin with pan because of the rate coding idea that's the inspiration um and in order to make this vae work we need to do two things first of all we need to introduce an equivalent reparametrization trick uh for the poan case we also need to compute the K term we're going to do both of those in the next few slides all right so as I mentioned now we just take the prior and and posterior we're going to replace gaussin with with pan and so gaussians need both mean and variance you have to Define both a mean and a variance to have a gaussian distribution but for pan you only need one number that's just the rate parameter and and this is the PDF of a pulan distribution give me a rate and I have this distribution so we have this prior rate and the posterior rate which depends on X and and this could be you know any general you can you can learn the prior rates you can you know output any some some generic posterior rate but here is where we add the predictive coding assumption we say that okay look let's call the prior rates just R and interpret those as some representation units that are just maintaining a prediction or anticipation of what is coming what kind of sensory information is is coming their way and then let's assume that the posterior rates are actually just um the a modulation of those existing rates so we just say take R and the encoder only computes some quantity we call it delt r that is a function of the specific stimulus shown to the encoder and then you you obtain the posterior rates by multiplying these uh two together and and this is just an illustration of the idea so you have this input it is processed by some encoder to produce Delta R and it is understood that this Delta R is going to interact multiplicatively with existing prior rates and then once you have this you know multiplication you're going to now you have your posterior distribution this is your approximate posterior and then once you sample from this um you're going to get discrete P counts because now you have Plus on in the gaussian case you would get continuous values but now we're going to get discrete integers which is Illustrated over here like for example let's say we have five neurons in the latent space we sample from this posterior we have a vector of 1 3 0 0 one so we take that Vector we pass it through the encoder and the encoder learns to map it back to whatever image that was shown and um these are all learnable parameters so the decoder Network the encoder Network and the prior rates they're

all learnable parameters so I want to emphasize there's like a bunch of difference um between pulan and gaussian bees the first difference is okay it's pulan it's not gaussian the other difference is prior is learned and and then the last difference is that the these prior rates are parameters that are shared between both the prior and the approximate posterior now let's talk about the POS onization trick we want to be able to uh when we sample event counts or Spike Counts from this Plus on distribution we want to be able to um differentiate through this operation a so how can we do that in order to show you the um the intuition behind our rization trick let's actually focus on the process that generates event counts so if if Z is some event count that is distributed according to Pon we know that um there are some inter event intervals that are distributed through according to this exponential distribution what delta T is is that suppose one event has happened um how how long do you have to wait until you observe another event those weight times intervent intervals they're distributed according to this exponential distribution with parameter Lambda and and this is just a well-known property of Plus on distribution we're going to use this we're going to exploit this fact to you know introduce our our trick all right so now suppose we want to actually just run this process um sample many times from this exponential distribution and then um in a finite window of observation time of 1 second we want to see how many events would we get so when we sample from this exponential maybe we get first uh time weight time is Delta T1 so we wait this much until we observe the first event we sample again it takes this much time to observe the second event again third event but the fourth event doesn't really occur between zero and one which is our previously defined observation time therefore this doesn't count so we have a total of three events because the sum of these uh three Delta T's is less than one so um sampling these from exponential and and then counting them like that like I like we did is equivalent just sampling from posant and getting like number three as our Spike count so we can actually just sample these numbers in parallel from the exponential distribution we compute their cumulative sum uh which is visualized over here and you can just apply the thresholds and say all right whatever number of cumulative sums that there's less than one let's take that as our um Spike count and here which means we're going to have three events and and this is not going to count and any other delt T's after this is not going to count because it's just going to add over here it's it's it's extends beyond our weight time so again we have total event counts of three it turns out that you can just take this and turn it into an algorithm um so the input to the algorithm is just some rates that are produced by the encoder Network then we have uh we have to specify how how many exponentials samples that we want to generate

there's a temperature that that controls the sharpness of the thresholding so if you want to threshold put a hard threshold of like anything less than one uh that's not going to work you have to um approximate that with some sigmoid uh that's what we have over here okay so that's our reper metrization trick and um and we hope that this this trick you know it can you know be used beyond the realm of vaes for example in spiking real networks uh people deal with this issue all the time you know it's it's really difficult to work with discrete stochastic variables so we're hoping that this algorithm will find applications elsewhere as well but now let's talk about um the loss function derivation as a reminder we have our prior given like this and the posterior given like this and our goal is to compute the K ter so let's first start by just definition of the KL term um for any distribution q and P the KL between q and P is um this expectation over Q of log of Q / P so so far this is just general definition but now we're going to plug in the actual poson distributions that we have so just put that over here we Define Lambda to be R * Delta R it's just our posterior rate and this is what Plus on distribution looks like so I just replace q and P and immediately we see that Z factorials cancel and we can actually just um write this term as follows it's just log of Lambda over R to the^ Z Plus log of e to the minus Lambda plus r and then Z comes down Lambda over R is Delta R so you're going to just end up with Z log Delta r and log of e to something is is very simple it's just you know you bring the exponent down and in this entire expression the only thing that that has an expectation is z everything else is a constant so it comes out and the expectation of z uh over Q is just it's the mean and the mean of poson distribution is just the rate so Z is going to be replaced by Lambda which is the rate of Q and that's we get here so R minus Lambda plus Lambda log Delta R and again we just take R take it out and it's like 1 minus Delta R plus Delta R log Delta R so we see that nicely um by having you know by virtue of choosing this kind of predictive coding like assumptions our K term nicely factorizes to a term that depends on prior and another term that depends on the modulation that comes from the encoder and then we Define this whole thing in parentheses to be F so that's f is 1 minus this plus whatever and if you plot it it looks like this so Delta R is by definition always positive it's greater than zero and um if Delta R is one it says like okay do not modulate anything let's just keep the prior rate then this F becomes zero and if Delta R is really large you you pay the cost of you know it it increases the magnitude of f of Y so okay so this scale term was just for one neuron um if you have K statistically independent neurons K being the dimensionality of our latent space you can just easily extend this derivation to see that the KL term is just the sum of of these terms and now this is the interesting part we're going to interpret this term as a metabolic

cost term the reason is are you know the the prior firing rates they are positive values by definition and we ended up with some term F that also is positive by definition so you have a positive term multiplied by another positive term so this thing whole thing is is just a positive value and we want to minimize it and we can minimize this for example by just putting zeros in all of our prior rates like all of these these neurons are silent they're not doing anything and and that's why um we are interpreting this as a metabolic cost because you know producing Action potentials um is is a very costly metabolically costly operation so the neurons in the brain they don't want to fire Action potentials unless it's absolutely necessary to encode some information or communicate something that's why U this is very interesting that it just comes out of the math and if you just put everything together this is just the the loss of Plus on vae the first term is the Reconstruction term um it's just the L2 Norm of X input minus reconstruction this is like decoder Network applied to one sample which is drawn from the posterior uh so this is the Reconstruction plus this term which is comes from the KL term and this is very curious because now we have some some term that says like reconstruct the image and another term that says do it while uh optimizing or or minimizing neural resource use and this is reminiscent of sparse coding which I will introduce here so in 96 um old s and field introduced sparse coding they said okay let's imagine there are some images X and some latent variable z um or like let's say neural activation z in their case um and let's assume that we represent these images as a linear sum of some basis elements from a dictionary fi while we're representing these images let's also try to minimize the magnitude of neural activity which is this L1 term over here and just um you know minimize this whole loss that'll give you sparse coding spse because you know if you if you apply this term it's going to push a lot of these uh neural activities to zero it's going to give you a sparse activity vector and and this really resembles what we get from pvee and in order to actually make this connection more clear we just take this General decoder term that appears in the loss we also assume we have a linear decoder just like sparse coding and it turns out in that case you have a closed form solution for the law um I'm not going to derive this it's in the paper if you're curious but you know there is a close form solution um in in the actual you know full loss there is this expectation value which you can compute it gives you this final loss and again we see this this correspondence nicely so there is one term that says um Faithfully represent images with some linear decoder um and then there are two terms here actually it's as opposed to just one there are two terms that push your firing rates to low values so this diagonal of Pi transpose Pi is positive by definition and Lambda is positive so this thing needs to minimize therefore it forces Lambda

to minimize and again just a reminder that this is coming from the K term and we interpret it as a metabolic cost ter okay so let's just pause a little bit here he started from all those um three Neuroscience Inspirations we said we want to build a model that does um you know perceives visual inputs we want to build in um perception as inference rate coding and predictive coding all of those in the model it gives us it um it gave us the the Bon vae idea we derived the loss and we saw that uh it resembles spse coding so it's just U gifted To Us by the theory now we want to test this we want to say okay the loss resembles sparse coding but let's actually just train some models and see if we get sparse coding like results that's what we're going to do next experiments oh yeah this is okay so this is the architecture of um linear PVE so you have again some image is encoded into delt R multiplied by R to give you this posterior and importantly now the decoder is just a linear layer that's what this is about and another uh important assumption that um is Central to sparse coding is that you have to have an over complete latent space so you have many more latent Dimensions or many more neurons than you have pixels so m is the number of pixels of the input image K is your latent dimensionality now we're going to train this model and compare it to sparse coding but also um in order to understand what component of the pvee might you know um you know reproduce Spar SC we're going to compare to Alternative vae models so we have discrete vaes we have pvee over here which is what we introduced and we're going to use um categorical vae uh and there are continuous va gaussian va and llas VA and the reason we chose llas to to include in in this uh in this table is that people have shown before us that if you have a lloss distributed prior you can also get sparse coding like results so we want to test which one of these models is actually uh more sparse coding like if you will and then we're going to compare to actual you know um very well established standard sparse coding algorithms one is the locally competitive algorithm from 2008 it's a biologically inspired algorithm that U uh optimizes the sparse coding loss and another algorithm is uh ISTA these are both very standard and uh well established um as well as the gaussians so you're going to compare to a bunch of models all right so if we train this pvee on linear pvea on natural image patches this is what we get these are the decoder weights and um if you're not if you're in Neuroscience in visual Neuroscience you already know that these are we refer to these are like the Bor patches and and um these resemble the selectivity in the primary visual cortex of monkeys and and mice and and humans so so this is very biology like and and for the first time these types of Gabor patches they they emerg from the sparse cating models which we are we were able to reproduce with pvea and if I show you the other models it's not actually trivial so let's go from

left so we have gaan VA over here I'm not sure if you can see but um there's a lot of noise and there's only like last three rows uh look you know they they they look like PCA and and that's expected because a linear gaing vae is theoretically equivalent to probabilistic PCA now we have also llas VA down here um there are some Gabor that emerge from the llas VA but again there's a lot of dead neurons which I will Define what dead neurons means and I I'm going to show how we quantify them but you know for now let's just visually inspect these learned decoder rates this is the posan vae and this is the categorical vae categorical VA is discrete so it also learns some Gabor but it's very limited so um that there's a lot of noisy elements and certain spatial frequencies are missing and certain orientations are missing as well from the category and finally in the last column we have these results from the standard sparse coding algorithms these are the best you know uh game in town if you want to learn sparse coding like U dictionary elements and we see that among the vae models um posan V actually is the is the like most similar one to these standard sparse coding algorithms so in conclusion because both U the loss function of a PVE just emerged to resemble sparse coding and also empirically we you know tested it and and also learns sparse coding like U results we conclude that it actually contains sparse coding as a special case why because you know we we made a linear decoder assumption so it's like PVE can be um General like it could be anything could be nonlinear encoder nonlinear decoder but if you make the decoder linear and make the Laten space over complete you're going to recover bar coding like results okay so here's um how I quantify number of dead neurons this is just a let me just first Define what the posterior collapse is this is a big kind of prevalent issue in vaes when a latent Dimension stops encoding information it's going to be always the posterior in that latent Dimension is going to be identical to the prior so there's no ation the K term is very small or zero and that's how we measure dead neurons and here this dashed line shows the gap between the post the kale of dead neurons and active neurons that's how we Quantified and uh we found that for the pvee uh across three different data sets it consistently had much larger number of active neurons compared to l and gaussian VA so it kind of automatically avoids this posterior collapse issue because of discreetness that's the the main cause of this observation okay so we want to actually do some other experiments um so far we just saw that if you start from those Neuroscience Inspirations you get a model that also you know recovers or or reproduces some other Neuroscience like uh models result like sparse coding but now let's just um be a little more General we want to evaluate PVE in a general representation learning setting that's what this is about all right so we want to test these UNS supervisor presentations on a downstream

classification task and we use nness for that purpose we're going to compare all those four vae models for this task the approach is that after you know the training is done extract encoder representations this means we push mest digits from the validation set through the encoder we extract the representations and we're going to do things with those all right and then once you have these representations sample a limited number of supervised labels so M this you know they have labels we know each image of a digits has a label so it's it's a supervised data set um but we going to look at subsamples like very small samples um of of supervised um to sets and and the reason is that you know um having labels is expensive so if a model can achieve good performance with a small subset of supervised labels is is a is a good model that's the idea then we're going to apply kir neighbor classifier um to to classify these these um digits and the idea of using KNN is that it's a non-parametric model so you don't need to learn anything on top of these representations and the output of a K&N classifier directly depends on the geometry of representation so it's going to tell us something about if there's something fundamentally different about the geometry of representations in one of these vaes like PV versus the others and then as you know as me measure of success we're going to report classification accuracy this is the results um you know we have different plat dimensionalities we have different models there's a lot of details that I'm going to skip but let me highlight just one thing so let's look at this number so for 10 latent Dimensions with only 200 samples the pvee achieves accuracy of around 82% but gaan vae requires five times that number a thousand label samples to achieve a similar performance in this sense we claim that pvee is five times more sample efficient than the standard gaan vae and you know across the board it just add performs all these other models which shows that it learns useful representations uh that can be applied in Downstream tasks finally let's let's quantify sparsity uh we're going to use this measure from V and Galant this is called lifetime sparcity um zi is the response to I stimulus number of spikes in the pan and N is the total number of stimuli um just the intuition about this formula is that um if if if if a neuron only responds to a single stimulus and doesn't respond to any other stimuli the sparcity score is going to be one and if the neuron or the latent Dimension responds to all stimuli equally then the sparity score is going to be zero so zero means not sparse at all one means uh very sparse and we use that this is the the y- axis the the lifetime sparity of of these different um VA models and on the x-axis we have the Reconstruction error in mean squared error um you see that posan is very sparse but because of that sparcity it's uh has larger reconstruction error compared to let's say gaussian which is down here and it turns out if you apply some some Ru um to the gaussian it's going to increase disparity but not as

much as Plus on and then this curve over here came from fitting a sigmoid to a bunch of Pon vaes that were trained with different beta so if you remember beta is a parameter that um uh kind of changes the trade-off between the Reconstruction term and the KL turn so if you increase beta you're going to get much sparer results but uh at a cost of larger reconstruction errors and if you if you decrease the beta you're going to get less sparse results but but better reconstruction so that kind of traces uh rate Distortion tradeoff finally um one thing that we Noti is that there is still a large amortization gap for the pul and I'll explain what that means so if you compare pan with those standard uh sparse coding algorithms including LCA which is this dark grade and ISTA which is this light grade we see that they achieved the same level of sparcity roughly uh but posang still has um a larger reconstruction error but you know what what happened here like what is why is um LCA much better than pan VI so to test this we we thought that you know maybe the dictionary elements learned by pan V are are are pretty good but the actual inference is is not very good the encoder architecture is not doing a good job in inferring the activations so what we did is we took the pan VA and we took um sorry the the dictionary elements learned by the Pang even applied LCA inference on those and that gave us this these purple dots which we also fit a sigmoid to and it turns out that you can actually just close the amortization Gap by applying a better inference which in this case is the LCA inference to just uh bring this over here so the conclusion from this slide is that the posan vae learns reasonable dictionary elements but it's encoder or the encoder architectures we tried in this paper they weren't adequate and there was this large geometrization gap that in the future work we hope to to kind of close this Gap by you know building better encoders maybe iterative encoders and so on okay let's just conclude um so we started from these three U Inspirations perception as inference rate coding predicted coding it gives us U this architecture and um it's interesting for many reasons because you know pan V encodes its inputs in discrete Spike counts therefore it's um more biologically realistic and this metabolic cost term emerged in the model objective for free you didn't have to put it by hand or anything and uh it actually revealed this connection to sparse coding which we verified empirically and then from a theoretical perspective uh it kind of Unites all these different ideas like predictive coding rate coding sparse Under the Umbrella of beian inference which is nice and then the takeaway message is that you know if you draw inspiration from centuries of Neuroscience research combine it with modern machine learning you might get a model that's more brain like than Alternatives so um and it outperforms Alternatives in key aspects like sample efficiency all right this is all I had for the slides um now if there are questions could go over there or maybe

I can show the code thank you that was awesome yeah how about the code and meanwhile for people who are watching write any questions and then I'll ask them thank you all right um yeah so I actually haven't prepared anything specific to say about the code but I'm just going to riff through you know what what exists um hopefully some some people might find this inspiring okay so this code is actually not online right now it's not available but because the paper is under review but uh once the paper gets out um I'm going to make this code publicly available so the maybe the more more important uh part of the code is that you go in the code in the vae you have va. py which has all the implementations of different vaes that we discussed in in this talk so there's a base vae that um has all these different functions it encodes um inputs you know you can have a convolutional encoder you can have a linear encoder MLP encoder it's all implemented you know same with decoder you have convolutional linear MLP it computes the loss which is just the msse um and you can also compute the exact loss when you have a linear decoder um so the KL loss is not implemented because we're going to have specific instances of vaes like L you know paulan VA gin VA Etc and they each have their own KW so that's that's not implemented over here in the base vae and so on there's there's a bunch of you know different functions so let me just show you the actual implementations so posan V is over here um so in the forward Direction you have this this infer function that accepts X which is an image temperature and it it computes so the self. log rate this is the prior rates these are parameters that are learned along with every other parameter and you have this log R um then you have log Dr Dr is just a delt R that's the output of the encoder where you encode the image as an output you get log Dr and then um skip this part then you compute you you make this distribution and and just to remind yourself that this is just Plus on which I will show you later it's defined somewhere else and then you you make this distribution you provide the log rates this is the NX if you remember from the reparameterization um slide you have to provide how many exponential samples you're going to draw you know it's it's adaptively computed somewhere else in the code I'll tell you where later you provide the temperature and then you have a distribution which you know this is going to be the infer function provides you with the distribution that's your approximate posterior and then you sample from it and get spikes you decode those spikes that's your reconstruction that's it there's other functions that I'm not going to go but maybe like let's have a look at the K term so um you can easily you know understand this you know if you understood the the slide deriving the kale term in theory then you would understand this term um maybe I'll I'll show you quickly distributions so we have also this based distributions

which contains our implementation of the pon distribution and the reparameterization trick for it um there we go it's here yeah so you provide it with log rates um in initialization you provide the temperature which is the thresholding temperature number of exper number of U exponential samples and the clamp is just basically says if the rate is larger than e to the this you know it applies a soft clamp which is e to the this value so it applies an upper bound on the rates this is for the you know stability of training it's like U not very important but the important part is here this function that's our reparametrization algorithm okay so first of all you initialize this exponential distribution in this init function so you you get log rates apply the soft clamp and then you define rates as this just uh exponentiate log rates plus this really tiny value so that it's non zero um because if it's zero then this exponential is going to raise an error it has to be uh slightly above zero so you define this exponential distribution which takes the rate as input and here I'm just borrowing from P torch so this uh if I go up I'm importing oh it's not over here I imported somewhere else but basically um where is it this is the torch. distributions I'm I'm just importing it as thiss so this exponential is exactly pytorch's implementation all right once we've initialized this self. X object then you can actually just perform pre parameterized sampling so X is just the first you know all those Delta T's that I showed in the slide these are your x's and you have your sample n times and then you perform this cumulative sum to get the times and then if temperature is greater than zero you have this soft indicator which is just sigmoid you know if it was temperature was Zero then you would have have this hard indicator which is just times less than one if if you take this operation if you make it continuous you will get this basically that's the idea once you have this indicator function then you can sum them in the First Dimension to get your Spike counts all right so that's that's for the PA on but I want to also mention this um continuous VA because this is a more General implementation that you can use for any you know any vae that uses a continuous distribution like in our case we used gaussian and llas but you can actually take this implementation and Implement easily build your own continuous VA like Ci or any other distribution you like as long as you can you can reparameterize it which a lot of pytorch you know implementations they have this function called R sample which is per sample but again there you know this is a very Sim similar idea you encode some features take some X encode it into this feature H and then because for these continuous VES you need both the mean which is location and the variance which is log scale so you just divide H into two parts you get these parameters and then you you know construct this posterior distribution and then return it that's what infer does it always G gives you the posterior distribution once you have

the posterior distribution you can R sample or reparameterized sample to get your latence U ignore this part you know if you can actually just apply an activation function this is the gaussian pr Ru that I I kind of skipped that didn't really talk about it but that's that's there and then you can decode and Y as your reconstruct so so this this continuous VA implementation is is fairly complete you can in order to build your own that's all you have to do so just say Okay I want to make a gaussian VA I'm gon to inherit continuous VA object I'm going to say okay self that this should be normal that's it and for llas again you inherit the continuous V and just say the distribution should be L plus and you can do this for any other continuous uh distribution maybe um that you know these are these are py Tores implementation so any continuous distribution implemented by pytorch that has reparameterized sampling um you can you can use this to build your own vae and then we have this categorical um which I'm going to skip but you know we use the again P torches implementation and okay so so this this um this module defines vaes and um there's another module that trains them so let me just briefly go over that again there's a bunch of code that just defines base trainers there's a lot of details here that maybe not very interesting but you have this trainer that accepts all these different vae types poson gaussian categorical loss and it goes through this is just one iteration um there's a validation function all of that and it's it's fairly complete so there's a bunch of um arguments that you have to provide to be able to run this code but because I spent a lot of time to make it intuitive it's going to be just a single line if you want to train your own PVE um you can just run uh that with a single line like you know train vae you specify the architecture in one string you specify the data set and um and the GPU that you want to train on and then then you can actually do that if you want but yeah this this code is not available right now when I put it online I'm going to make some documentation and you know put some scripts that make make it easy and intuitive to just train your own model if you're interested there's a lot more but I think at this point I'm going to stop yeah there's there's a bunch of other stuff um that is secondary yeah awesome okay while I um crop back the screen maybe just let's start with how did you get to this problem were you pursuing Pon distributions and this came up or how did you kind of um in your research path come to this approach oh so the the real initial motivation is just you know um we we did another paper um last year on a gaussian vae that was hierarchical and we found that that was um we trained that gaussian vae that gaussian hierarchical VA on a motion data set and we found that um it learned latent representations that were very aligned to real biological neurons from the monkey brain so we have another data set uh from um the data set was they showed some moving dots to monkeys and they

recorded from their Mt which is a brain region that you know um that is responsible for motion perception anyway we found that these vaes actually have a really high chance of being brain brain like and that gave us motivation to to think about okay how we can make them even more brain like so Gan is just en code inputs in continuous values that are not you know um constrained by any any means but we know that that's not how brain works we know that neurons produce spikes that's how they communicate so we said like okay vaes have potential but how how can we make them better that was really the initial motivation awesome well it just to kind of recap a little bit of how I saw this often parameters in biological or cognitive modeling are modeled as continuously dynamically varying because there are certain simplifying assumptions however the empirical data be it neural spiking or events or like ants foraging how do you bridge the discrete events to The Continuous and the um mathematical equivalence which is well known between the event arrival distribution with the Brant and the waiting time I saw that as a way that you implemented with the statistical methods to and also in doing so generalize like what was relevant for the base model for continuous or discreet and then how can that be articulated with the statistical packages so that there's good course handles to explore these learning settings I think that's actually very interesting and and and deep point you know um even though neurons communicate with spikes or discrete Action potentials we when we are modeling them you can actually think as um you know the actual underlying Dynamics could happen in a continuous space like for example we had these log rates you know those log rates are continuous values between minus infinity and plus infinity really I mean even though in reality it doesn't go that far but they're defined in if you have n neurons log rates are defined in RN right and then uh you can assume that the model is just doing some trajectory like the brain is doing going through some trajectory in that log rate space and in each given time you can actually take this that log rate and then map it to some posterior by computing the exponential of that to get the rates and then draw samples and in that sense you can actually see that a continuous underlying dynamical system can be mapped into a discrete observation through through paon and also that um every time point in the Dynamics of the brain also corresponds to some posterior inference so these Concepts I think can be nicely you know reconciled with this View yeah that was great and to kind of connect it to active inference and a little bit of the precursors there with SPM statistical parametric mapping the whole setting is there's underlying neural activity hidden State external State and then there's noise plus signal representing a static mapping between the hidden and the observable what you had as x and z so that's like the

partial observability sit situation and then that's the experimental situation but you don't get to observe neural rates so like in the SPM textbook and the kind of modeling you model it with the Predator pre type models or with dynamical attractors but you stay in the purely dynamical and then that doesn't let you get to the rate coding but also people have integrated those models however in more of a um two-step process rather than the onestep UniFi objective of the auto encoder yeah so that's pretty interesting so because rate is in a sense the rate itself is a latent variable right we we never observe it the it's a it's a madeup concept really by the theorists we say okay there if there is an underlying rate and that is producing these observed spikes you know in the brain then then you you can actually try to infer the rate by observing many spikes over many trials and so on so in that sense yeah rate itself is a latent variable um but that that maybe you shouldn't go there because that like adds another layer of complexity one question I had about rate was if you're going to model it you kind of can't get away from it as a underlying dynamical variable because there's going to be variability in delta T variance amongst arrival time so maybe it's an metronome but in the non- metronome case there's going to be kind of like fast and slower intervals so that's kind of the common filter question which is or or ARA or any time series ultimately how smoothly should you interpolate that like with the splines with what method will the data kind of have a relationship with a curve that has the smoother differential properties that ultimately allowed the close form solution and the simple simple algorithms mhm makes sense um okay I'll see if anyone asks a question but um just one question now and then I'll ask maybe one or two more um like where do you see this being useful slash applied now so we are actually currently building um based on this observation that there was a lot large amortization gap between Clon vae and this uh kind of relatively old sparse coding algorithm from 2008 we one conclusion we drew from that is that you know we need iterative inference so right now the inference is just one path you take one image you you infer something Del to compute Delta R and then you just get the posterior but now suppose you have a sequence you have a video rather than images and you want to just keep iteratively making your inference better and better such that your um let's say your posterior at time T informs your prior at time t plus one that's like very essential to beian thinking so we're kind of right now developing that model and our hope you know very long-term hope is that you know when we add that component and maybe we add hierarchical structure to the latent space then we get like something that is truly brain-like and it reproduces what we've observed what we've learned about visual processing in the last like you know 50 years so if you get that model then you can perform in silico experiments in it

learn something about brain like visual processing and so on so that's kind of roughly the the trajectory we imagine awesome is there anything else you want to add otherwise I think this was amazing on the theory and really seems use use ful with the sample efficiency all these other features yeah um I guess that's it yeah thanks for having me this was really fun awesome okay I'll just read one comment from someone wrote great work thanks to ACM and AG ha Hadi for sharing looking forward to find out what a different inference scheme could do to the results and the effect it could have on the metabolic cost okay yeah thanks for the comment yeah cool com that in mind all right thank you see you all right thank you bye